Programación multihilo I-B

Programación de servicios y procesos

Contenidos:

1) Finalizar un hilo
2) ExecutorService

Finalizar un hilo

La forma natural de que muera o finalice un hilo es cuando termina de ejecutarse su método run(), pasando al estado 'Muerto'. Una vez que el hilo ha muerto, no lo puedes iniciar otra vez con start(). Si en tu programa deseas realizar otra vez el trabajo desempeñado por el hilo, tendrás que:

- ➔ Crear un nuevo hilo con new().
- ➔ Iniciar el hilo con start().

Y ¿hay alguna forma de comprobar si un hilo no ha muerto?

No exactamente, pero puedes utilizar el método **isAlive()** de la clase thread para comprobar si un hilo está vivo o no. Un hilo se considera que está vivo (alive) desde la llamada a su método start() hasta su muerte. **isAlive()** devuelve verdadero (true) o falso (false), según que el hilo esté vivo o no.

Para finalizar un hilo hay que “interrumpirlo”, esto es llamar al método **interrump().** Esto dispara la excepción *InterrupedException* en la que codificamos la salida del hilo.

El método **stop()** de la clase thread (actualmente en desuso) también finaliza un hilo, pero es poco seguro. No debes utilizarlo. Te lo indicamos aquí simplemente porque puede que encuentres programas utilizando este método.

Ejemplo parar un hilo. Bomba

Con uno de los ejercicios propuestos en el tema vamos a ver cómo se puede controlar un hilo y pararlo.

El código de este ejercicio está en UT2_1_Ej5_Bomba

La clase ArtifieroBomba extiende de la clase hilo (thread), tiene un constructor que recibe un booleano para saber qué método se arranca con el hilo. Hay 2 métodos.

El método contador que simula una cuenta atrás de 10 y que al final “estalla” la bomba.

El método artificiero que simula lo que tarda un artificiero en desactivar la bomba con un número aleatorio entre 9,5 y 10,5 segundos.

Ejemplo parar un hilo. Bomba

```
public void contador() {
    try {
        int j =9;
        for(int i = 0; i < 10; i++) {
            System.out.printf(String.valueOf(j-i)+" %n");
            this.sleep(1000);
        }
            System.out.printf("PUUUUUUMMMMMMMMM%n");
    }catch (InterruptedException e) {
        // se lanza esta excepción cuando interrumpimos el hilo
        // Aquí colocamos el código que queremos que se ejecute.
        // En este caso, irnos
        return;
    }
}
```

En el control de la excepción de interrupción, nos salimos del programa.

Con return salimos del hilo, con System.exit(0) salimos del programa (como ya sabemos)

Ejemplo parar un hilo. Bomba

```
public  void artificero() {
    try {
        Integer numero = (int)(Math.random()*(1050-950+1)+950);
        System.out.printf("El artificiero comienza a desactivar la bomba%n") ;
        this.sleep(numero*10);
        System.out.printf("El artificiero ha desactivado la bomba%n") ;
    }catch (InterruptedException e) {
        // se lanza esta excepción cuando interrumpimos el hilo
        // Aquí colocamos el código que queremos que se ejecute.
        // En este caso, irnos
        return;
    }
}
```

En el control de la excepción de interrupción, nos salimos del programa.

Ejemplo parar un hilo. Bomba

```
public static void main(String[] args) throws InterruptedException {
    ArtifieroBomba artificiero = new ArtifieroBomba(false);
    ArtifieroBomba bomba = new ArtifieroBomba(true);

    artificiero.start();
    bomba.start();
    do {
        try{
            Thread.sleep(50);
        }catch (InterruptedException exc){
            System.out.println("Hilo principal interrumpido.");
        }
    } while (bomba.isAlive()&&
            artificiero.isAlive());

    if (bomba.isAlive()) {
        bomba.interrupt();
        System.out.printf("La bomba ha sido desactivada%n");
    } else {
        artificiero.interrupt();
        System.out.printf("El artificiero ha muerto%n");
    }
}
```

Controlamos que uno de los hilos sigue vivo. Cuando uno no lo esté, saldremos de este while

En función de qué hilo haya acabado, paramos el otro lanzando la excepción de interrupción del hilo que paramos

Pool de hilos

Cuando trabajamos con aplicaciones tipo servidor, éstas tienen que atender un número masivo y concurrente de peticiones de usuario, en forma de tareas que deben ser procesadas lo antes posible. Al principio de la unidad ya te indicamos mediante un ejemplo la conveniencia de utilizar hilos en estas aplicaciones, pero si ejecutamos cada tarea en un hilo distinto, se pueden llegar a crear tantos hilos que el incremento de recursos utilizados puede comprometer la estabilidad del sistema. Los *pools de hilos* ofrecen una solución a este problema.

Y ¿qué es un pool de hilos? Un pool de hilos (thread pools) es básicamente **un contenedor** dentro del cual se crean y se inician **un número limitado de hilos**, para ejecutar **todas las tareas de una lista**.

Pool de hilos

Para declarar un pool, lo más habitual es hacerlo como un objeto del tipo ExecutorService utilizando alguno de los siguientes métodos de la clase estática Executors:

- ✓ **newFixedThreadPool(int numeroHilos)**: crea un pool con el número de hilos indicado. Dichos hilos son reutilizados cíclicamente hasta terminar con las tareas de la cola o lista.
- ✓ **newCachedThreadPool():** crea un pool que va creando hilos conforme se van necesitando, pero que puede reutilizar los ya concluidos para no tener que crear demasiados. Los hilos que llevan mucho tiempo inactivos son terminados automáticamente por el pool.
- ✓ **newSingleThreadExecutor():** crea un pool de un solo hilo. La ventaja que ofrece este esquema es que si ocurre una excepción durante la ejecución de una tarea, no se detiene la ejecución de las siguientes.
- ✓ **newScheduledExecutor() :** crea un pool que va a ejecutar tareas programadas cada cierto tiempo, ya sea una sola vez o de manera repetitiva. Es parecido a un objeto Timer, pero con la diferencia de que puede tener varios threads que irán realizando las tareas programadas conforme se desocupen.

ExecutorService

Hasta ahora, toda la creación de hilos ha sido “manual”. Cada vez que nos ha hecho falta un hilo hemos instanciado manualmente un objeto Thread, especificando el código que se ejecuta mediante un Runnable o extendiendo de la clase Thread.

Sin embargo, esta forma de desarrollar aplicaciones Java con concurrencia no es la más recomendada. Java nos recomienda usar ExecutorService en nuestros desarrollos que ejecutan concurrencia en lugar de crear los hilos directamente. Principales razones:

ExecutorService

- **Desacoplamiento de la creación y la ejecución**: nosotros indicamos qué se hace, ExecutorService se encarga de decidir cómo lo hace ya que no siempre creará hilos nuevos sino que reutilizará aquellos que estén disponibles.
- **Reutilización de Hilos (Thread Pooling)**: ExecutorService implementa el concepto de pool de hilos. Esta funcionalidad te permite limitar los hilos a utilizar:
  - Puedes configurar un ExecutorService para tener un número fijo de hilos (por ejemplo, con Executors.newFixedThreadPool(int n)). Esto evita la posibilidad de que se creen demasiados hilos, lo que podría agotar los recursos del sistema y provocar un fallo
  - Si tienes muchas tareas cortas que quieres ejecutar de forma concurrente, el sistema operativo pasará mucho tiempo creando y cerrando hilos.
  - Cuando una tarea termina, el hilo no se destruye, sino que regresa al pool para esperar otra tarea. Esto reduce drásticamente la sobrecarga (overhead) y mejora la capacidad de respuesta y el rendimiento general de la aplicación.
  - Ejemplo UT3_7_PoolHilos
- **Gestión de resultados**: aunque no lo vamos a ver en el curso, ExecutorService te permite recuperar el resultado de la ejecución de un hilo. Para ello hay que usar la interfaz Callable en lugar de Runnable

ExecutorService

Hasta ahora hemos estado ejecutando hilos que o bien acababan sólos o bien desde el main parábamos algún hilo en función de alguna circunstancia (Bomba).

A partir de ahora, y dado que es la opción que Java recomienda, los hilos se deben abrir con ExexutorService. Para ellos se implementaran las clases hilos bien extendiendo de Thread o bien implementando Runnable (funciona con ambos) y utilizaremos los métodos submit / execute para lanzar los hilos:

- **execute**(): es un método de la clase Executor. Ejecuta un objeto runnable y en ningún caso devolverá resultado.
- **submit**(): es un método de la interfaz ExecutorService que a su vez deriva de Executor. Con el método submit(), y un objeto Calleable, podríamos recuperar el resultado de un hilo

También utilizaremos el método **shutdwon**(). El método shutdown sirve para indicarle al servicio que se apague. Su funcionamiento es el siguiente:

ExecutorService

- A partir de ese momento, el método submit no aceptará nuevas tareas. Si tratas de llamar a submit luego de llamar a shutdown y empezar a apagarlo, te lanzará una excepción porque no puede aceptar tareas en este momento.
- Se permite que las tareas que están ahora mismo en ejecución terminen. El método shutdown va a dejar que terminen.
- Una vez las tareas que sí están ejecutándose terminen, el servicio será terminado también. Puedes saber si ha sido terminado usando el método **isTerminated**(), que devuelve falso hasta que un servicio que previamente ha sido apagado ya no tenga tareas en ejecución.
- También puedes usar el método **awaitTermination**(), que directamente bloquea tu hilo hasta que el servicio se termine. Puedes imaginarlo como el join() de un Thread, pero masivo. Esto es porque si has mandado docenas de tareas a un servicio, con este método podrás esperar que terminen todas.
- Si por lo que sea necesitas detener un servicio sin esperar a que terminen las tareas, puedes cancelarlas todas a la vez que apagas el servicio usando el método **shutdownNow**(). Cuando invoques este método, por lo tanto, es bastante probable que todos los hilos reciban una interrupción, para que terminen tan rápido como puedan.

Ejemplo UT2_1_TIC_TAC_ExecutorService

Ventajas del uso de ExecutorService?? https://oregoom.com/java/pool-de-hilos/#google_vignette